

```

"""
ScholarDeals Ghana Telegram Bot
=====

Sells: Checker Cards + University Forms
Affiliate: GH¢25 commission per form sold via referral
Platform: Railway (PostgreSQL + Python)
"""

import os
import logging
import time
import psycpg2
from psycpg2 import pool
from functools import wraps
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import (
    Application, CommandHandler, MessageHandler,
    CallbackQueryHandler, ConversationHandler, filters, ContextTypes
)

# ——— Logging ———
logging.basicConfig(
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    level=logging.INFO
)

logging.getLogger("httpx").setLevel(logging.WARNING)
logging.getLogger("httpcore").setLevel(logging.WARNING)
logger = logging.getLogger(__name__)

# ——— Environment Variables ———
BOT_TOKEN = os.environ["TELEGRAM_BOT_TOKEN"]
ADMIN_ID = int(os.environ.get("TELEGRAM_ADMIN_ID", "8255451505"))
MOMO_NUMBER = os.environ.get("MOMO_NUMBER", "0530790707")
ACCOUNT_NAME = os.environ.get("ACCOUNT_NAME", "Frank Nsiah")
ADMIN_NAME = os.environ.get("ADMIN_NAME", "Luther King")
ADMIN_WHATSAPP = os.environ.get("ADMIN_WHATSAPP", "233530790707")
WHATSAPP_CHANNEL = os.environ.get("WHATSAPP_CHANNEL", "https://whatsapp.com/chann
WHATSAPP_GROUP = os.environ.get("WHATSAPP_GROUP", "https://chat.whatsapp.com/HZ
RESULT_LINK = os.environ.get("RESULT_LINK", "https://ghana.waecdirect.org")
DATABASE_URL = os.environ["DATABASE_URL"]
CHECKER_PRICE = float(os.environ.get("PRICE", "18.50"))

# Prices
FORM_PRICE_PUBLIC = 295.00

```

FORM_PRICE_TECHNICAL = 250.00

AFFILIATE_COMMISSION = 25.00

— Checker Types —————

```
CHECKER_TYPES = {  
    "1": "WASSCE Checker",  
    "2": "Nov/Dec Private Checker",  
    "3": "BECE Checker",  
    "4": "Placement Checker",  
}
```

— University Lists —————

```
PUBLIC_UNIVERSITIES = [  
    "UG (University of Ghana)",  
    "KNUST (Kwame Nkrumah University of Science & Technology)",  
    "UCC (University of Cape Coast)",  
    "UEW (University of Education, Winneba)",  
    "UDS (University for Development Studies)",  
    "UMaT (University of Mines and Technology)",  
    "UHAS (University of Health and Allied Sciences)",  
    "UENR (University of Energy and Natural Resources)",  
    "UPSA (University of Professional Studies, Accra)",  
    "USTED (University of Science and Technology, Kumasi)",  
    "UESD (University of Environment and Sustainable Development)",  
    "GCTU (Ghana Communication Technology University)",  
    "UniMAC (University of Media, Arts and Communication)",  
]
```

```
TECHNICAL_UNIVERSITIES = [  
    "ATU (Accra Technical University)",  
    "KsTU (Kumasi Technical University)",  
    "KTU (Koforidua Technical University)",  
    "CCTU (Cape Coast Technical University)",  
    "TTU (Takoradi Technical University)",  
    "HTU (Ho Technical University)",  
    "BTU (Bolgatanga Technical University)",  
]
```

— Conversation States —————

```
(  
    MAIN_MENU,  
    CHECKER_TYPE, CHECKER_QTY, CHECKER_PAYMENT,  
    FORM_CATEGORY, FORM_UNIVERSITY, FORM_PAYMENT,  
    STUDY_REQUEST,  
    AFFILIATE_MENU,  
    WITHDRAW_MOMO,  
)  
= range(10)
```

```

# — Database —————
db_pool = None

def init_db():
    global db_pool
    db_pool = pool.ThreadedConnectionPool(1, 10, DATABASE_URL)
    conn = db_pool.getconn()
    try:
        cur = conn.cursor()

        cur.execute("""
            CREATE TABLE IF NOT EXISTS checker_pins (
                id SERIAL PRIMARY KEY,
                checker_type TEXT,
                serial_no TEXT,
                pin TEXT,
                used BOOLEAN DEFAULT FALSE,
                order_id INTEGER,
                created_at TIMESTAMP DEFAULT NOW()
            )
        """)

        cur.execute("""
            CREATE TABLE IF NOT EXISTS orders (
                id SERIAL PRIMARY KEY,
                user_id BIGINT,
                username TEXT,
                full_name TEXT,
                order_type TEXT,
                checker_type TEXT,
                university TEXT,
                quantity INTEGER DEFAULT 1,
                total NUMERIC(10,2),
                status TEXT DEFAULT 'pending',
                file_id TEXT,
                referrer_id BIGINT,
                commission_paid BOOLEAN DEFAULT FALSE,
                created_at TIMESTAMP DEFAULT NOW(),
                updated_at TIMESTAMP DEFAULT NOW()
            )
        """)

        cur.execute("""
            CREATE TABLE IF NOT EXISTS study_requests (
                id SERIAL PRIMARY KEY,
                user_id BIGINT,

```

```

        username      TEXT,
        full_name     TEXT,
        request_text  TEXT,
        status        TEXT DEFAULT 'open',
        created_at    TIMESTAMP DEFAULT NOW()
    )
    """
)

cur.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id             BIGINT PRIMARY KEY,
        username       TEXT,
        full_name      TEXT,
        referrer_id    BIGINT,
        affiliate_balance NUMERIC(10,2) DEFAULT 0,
        joined_at      TIMESTAMP DEFAULT NOW()
    )
    """)

cur.execute("""
    CREATE TABLE IF NOT EXISTS withdrawals (
        id             SERIAL PRIMARY KEY,
        user_id        BIGINT,
        amount         NUMERIC(10,2),
        momo_number    TEXT,
        status         TEXT DEFAULT 'pending',
        created_at     TIMESTAMP DEFAULT NOW()
    )
    """)

conn.commit()
logger.info("Database tables created/verified.")
finally:
    cur.close()
    db_pool.putconn(conn)

def db_retry(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        global db_pool
        for attempt in range(3):
            try:
                return func(*args, **kwargs)
            except psycopg2.OperationalError as e:
                logger.warning(f"DB error attempt {attempt+1}: {e}")
                if attempt < 2:
                    time.sleep(1)

```

```

        try:
            db_pool = pool.ThreadedConnectionPool(1, 10, DATABASE_URL)
        except Exception:
            pass
    else:
        raise

    return wrapper

# — DB Helpers —————

@db_retry
def register_user(user_id, username, full_name, referrer_id=None):
    conn = db_pool.getconn()
    try:
        cur = conn.cursor()
        cur.execute(
            """
            INSERT INTO users (id, username, full_name, referrer_id)
            VALUES (%s, %s, %s, %s)
            ON CONFLICT (id) DO UPDATE SET
                username = EXCLUDED.username,
                full_name = EXCLUDED.full_name
            """,
            (user_id, username, full_name, referrer_id)
        )
        conn.commit()
    finally:
        cur.close()
        db_pool.putconn(conn)

@db_retry
def get_user(user_id):
    conn = db_pool.getconn()
    try:
        cur = conn.cursor()
        cur.execute("SELECT id, username, full_name, referrer_id, affiliate_balanc")
        return cur.fetchone()
    finally:
        cur.close()
        db_pool.putconn(conn)

@db_retry
def create_order(user_id, username, full_name, order_type, checker_type=None,
                 university=None, quantity=1, total=0.0, file_id=None, referrer_id=None):
    conn = db_pool.getconn()
    try:
        cur = conn.cursor()

```

```

cur.execute(
    """
    INSERT INTO orders (user_id, username, full_name, order_type,
        checker_type, university, quantity, total, file_id, referrer_id)
    VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
    RETURNING id
    """,
    (user_id, username, full_name, order_type,
        checker_type, university, quantity, total, file_id, referrer_id)
)
order_id = cur.fetchone()[0]
conn.commit()
return order_id
finally:
    cur.close()
    db_pool.putconn(conn)

@db_retry
def get_order(order_id):
    conn = db_pool.getconn()
    try:
        cur = conn.cursor()
        cur.execute("SELECT * FROM orders WHERE id = %s", (order_id,))
        return cur.fetchone()
    finally:
        cur.close()
        db_pool.putconn(conn)

@db_retry
def fetch_and_lock_pins(checker_type, quantity, order_id):
    """Atomically fetch and lock pins to prevent race conditions."""
    conn = db_pool.getconn()
    try:
        cur = conn.cursor()
        cur.execute(
            """
            UPDATE checker_pins
            SET used = TRUE, order_id = %s
            WHERE id IN (
                SELECT id FROM checker_pins
                WHERE checker_type = %s AND used = FALSE
                ORDER BY id
                LIMIT %s
                FOR UPDATE SKIP LOCKED
            )
            RETURNING serial_no, pin
            """,

```

```

        (order_id, checker_type, quantity)
    )
    pins = cur.fetchall()
    conn.commit()
    return pins
finally:
    cur.close()
    db_pool.putconn(conn)

@db_retry
def approve_order(order_id):
    conn = db_pool.getconn()
    try:
        cur = conn.cursor()
        cur.execute(
            "UPDATE orders SET status = 'approved', updated_at = NOW() WHERE id =
            (order_id,)"
        )
        conn.commit()
    finally:
        cur.close()
        db_pool.putconn(conn)

@db_retry
def reject_order(order_id):
    conn = db_pool.getconn()
    try:
        cur = conn.cursor()
        cur.execute(
            "UPDATE orders SET status = 'rejected', updated_at = NOW() WHERE id =
            (order_id,)"
        )
        conn.commit()
    finally:
        cur.close()
        db_pool.putconn(conn)

@db_retry
def add_affiliate_commission(referrer_id, order_id):
    """Credit commission to referrer and mark order as commission_paid."""
    conn = db_pool.getconn()
    try:
        cur = conn.cursor()
        cur.execute(
            "SELECT commission_paid FROM orders WHERE id = %s",
            (order_id,)
        )

```

```

        row = cur.fetchone()
        if row and not row[0]:
            cur.execute(
                "UPDATE users SET affiliate_balance = affiliate_balance + %s WHERE
                (AFFILIATE_COMMISSION, referrer_id)
            )
            cur.execute(
                "UPDATE orders SET commission_paid = TRUE WHERE id = %s",
                (order_id,)
            )
            conn.commit()
    finally:
        cur.close()
        db_pool.putconn(conn)

@db_retry
def get_pin_stock():
    conn = db_pool.getconn()
    try:
        cur = conn.cursor()
        cur.execute(
            "SELECT checker_type, COUNT(*) FROM checker_pins WHERE used = FALSE G
        )
        return cur.fetchall()
    finally:
        cur.close()
        db_pool.putconn(conn)

@db_retry
def add_pin(checker_type, serial_no, pin):
    conn = db_pool.getconn()
    try:
        cur = conn.cursor()
        cur.execute(
            "INSERT INTO checker_pins (checker_type, serial_no, pin) VALUES (%s,
            (checker_type, serial_no, pin)
        )
        conn.commit()
    finally:
        cur.close()
        db_pool.putconn(conn)

@db_retry
def get_pending_orders():
    conn = db_pool.getconn()
    try:
        cur = conn.cursor()

```



```

        cur.execute(
            "SELECT id, user_id, username, order_type, checker_type, university,
        )
        return cur.fetchall()
    finally:
        cur.close()
        db_pool.putconn(conn)

@db_retry
def get_affiliate_stats(user_id):
    conn = db_pool.getconn()
    try:
        cur = conn.cursor()
        cur.execute("SELECT affiliate_balance FROM users WHERE id = %s", (user_id
        balance_row = cur.fetchone()
        balance = balance_row[0] if balance_row else 0

        cur.execute(
            "SELECT COUNT(*) FROM orders WHERE referrer_id = %s AND status = 'app
            (user_id,)
        )
        referrals = cur.fetchone()[0]

        cur.execute(
            "SELECT COUNT(*) FROM users WHERE referrer_id = %s",
            (user_id,)
        )
        total_referred = cur.fetchone()[0]

        return balance, referrals, total_referred
    finally:
        cur.close()
        db_pool.putconn(conn)

@db_retry
def create_withdrawal(user_id, amount, momo_number):
    conn = db_pool.getconn()
    try:
        cur = conn.cursor()
        cur.execute(
            "INSERT INTO withdrawals (user_id, amount, momo_number) VALUES (%s, %
            (user_id, amount, momo_number)
        )
        withdrawal_id = cur.fetchone()[0]
        cur.execute(
            "UPDATE users SET affiliate_balance = affiliate_balance - %s WHERE id
            (amount, user_id)

```

```

        )
        conn.commit()
        return withdrawal_id
    finally:
        cur.close()
        db_pool.putconn(conn)

@db_retry
def get_all_users():
    conn = db_pool.getconn()
    try:
        cur = conn.cursor()
        cur.execute("SELECT id FROM users")
        return [row[0] for row in cur.fetchall()]
    finally:
        cur.close()
        db_pool.putconn(conn)

# ——— Keyboards —————

def main_menu_keyboard():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("📄 Buy Checker Cards", callback_data="menu_checker"),
         InlineKeyboardButton("🎓 Buy University Forms", callback_data="menu_form"),
         InlineKeyboardButton("📚 Study Materials", callback_data="menu_study")],
        [InlineKeyboardButton("💰 Affiliate Program", callback_data="menu_affilia"),
         InlineKeyboardButton("☎️ Contact Admin", url=f"https://wa.me/{ADMIN_WHAT)",
         InlineKeyboardButton("🌐 Join WhatsApp Channel", url=WHATSAPP_CHANNEL)],
    ])

def back_keyboard():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("⬅️ Back to Main Menu", callback_data="back_main")]
    ])

# ——— /start —————

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user = update.effective_user
    args = context.args

    # Check for referral code
    referrer_id = None
    if args:
        try:
            ref_id = int(args[0])
            if ref_id != user.id:

```

```

        referrer_id = ref_id
    except ValueError:
        pass

# Register user
existing = get_user(user.id)
if not existing:
    register_user(
        user.id,
        user.username or "",
        user.full_name or "",
        referrer_id
    )
# Notify admin of new user
ref_text = f" (referred by ID {referrer_id})" if referrer_id else ""
try:
    await context.bot.send_message(
        ADMIN_ID,
        f"New user joined: {user.full_name} (@{user.username or 'no usern
    )
except Exception:
    pass
else:
    register_user(user.id, user.username or "", user.full_name or "")

await update.message.reply_text(
    f"Welcome to ScholarDeals Ghana!\n\n"
    f"We sell:\n"
    f"  - Checker Cards (GH\u00a218.50 each)\n"
    f"  - University Forms (GH\u00a2295 / GH\u00a2250)\n"
    f"  - Study Materials\n\n"
    f"Earn GH\u00a225 for every form sold through your referral link!\n\n"
    f"What would you like to do?",
    reply_markup=main_menu_keyboard()
)

# — Main Menu Handler —————

async def main_menu_callback(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    data = query.data

    if data == "back_main":
        await query.edit_message_text(
            "What would you like to do?",
            reply_markup=main_menu_keyboard()

```

```

    )

elif data == "menu_checker":
    keyboard = InlineKeyboardMarkup([
        [InlineKeyboardButton("1. WASSCE Checker", callback_data="checker_1")
        [InlineKeyboardButton("2. Nov/Dec Private Checker", callback_data="ch
        [InlineKeyboardButton("3. BECE Checker", callback_data="checker_3")],
        [InlineKeyboardButton("4. Placement Checker", callback_data="checker_
        [InlineKeyboardButton("⬅ Back", callback_data="back_main")]],
    ])
    await query.edit_message_text(
        f"Select checker type (GH\u00a2{CHECKER_PRICE:.2f} each):",
        reply_markup=keyboard
    )

elif data == "menu_forms":
    keyboard = InlineKeyboardMarkup([
        [InlineKeyboardButton(f"🏛 Public Universities (GH\u00a2{FORM_PRICE_F
        [InlineKeyboardButton(f"🔧 Technical Universities (GH\u00a2{FORM_PRIC
        [InlineKeyboardButton("⬅ Back", callback_data="back_main")]],
    ])
    await query.edit_message_text(
        "Select university category:",
        reply_markup=keyboard
    )

elif data == "menu_study":
    await query.edit_message_text(
        "Tell me what course and level you need study materials for.\n\n"
        "Example: Core Mathematics, SHS Level 2\n\n"
        "Type your request now:",
        reply_markup=back_keyboard()
    )
    context.user_data["state"] = "study_request"

elif data == "menu_affiliate":
    await show_affiliate_menu(query, context)

# — Checker Cards Flow —————

async def checker_type_callback(update: Update, context: ContextTypes.DEFAULT_TYP
    query = update.callback_query
    await query.answer()
    data = query.data # e.g. checker_1

    checker_key = data.split("_")[1]
    checker_name = CHECKER_TYPES.get(checker_key)

```

```

if not checker_name:
    return

context.user_data["checker_type"] = checker_name
context.user_data["state"] = "checker_qty"

await query.edit_message_text(
    f"You selected: {checker_name}\n"
    f"Price: GH\u00a2{CHECKER_PRICE:.2f} per card\n\n"
    f"How many cards do you want? (Enter a number, e.g. 1, 2, 3...)"
)

async def forms_category_callback(update: Update, context: ContextTypes.DEFAULT_T
    query = update.callback_query
    await query.answer()
    data = query.data

    if data == "forms_public":
        unis = PUBLIC_UNIVERSITIES
        price = FORM_PRICE_PUBLIC
        context.user_data["form_category"] = "public"
    else:
        unis = TECHNICAL_UNIVERSITIES
        price = FORM_PRICE_TECHNICAL
        context.user_data["form_category"] = "technical"

    context.user_data["form_price"] = price

    # Build university keyboard
    buttons = []
    for i, uni in enumerate(unis):
        buttons.append([InlineKeyboardButton(uni, callback_data=f"uni_{i}")])
    buttons.append([InlineKeyboardButton("⬅ Back", callback_data="menu_forms")])

    context.user_data["unis_list"] = unis

    await query.edit_message_text(
        f"Select your university (GH\u00a2{price:.0f} per form):",
        reply_markup=InlineKeyboardMarkup(buttons)
    )

async def university_select_callback(update: Update, context: ContextTypes.DEFAUL
    query = update.callback_query
    await query.answer()
    data = query.data # uni_0, uni_1, etc.

    idx = int(data.split("_")[1])

```

```

unis = context.user_data.get("unis_list", [])
if idx >= len(unis):
    return

university = unis[idx]
price = context.user_data.get("form_price", FORM_PRICE_PUBLIC)

context.user_data["university"] = university
context.user_data["state"] = "form_payment"

await query.edit_message_text(
    f"University Form: {university}\n"
    f"Price: GH\u00a2{price:.2f}\n\n"
    f"Make payment to:\n"
    f"MoMo Number: {MOMO_NUMBER}\n"
    f"Name: {ACCOUNT_NAME}\n"
    f"Amount: GH\u00a2{price:.2f}\n\n"
    f"After payment, send a screenshot of your payment here."
)

# — Message Handler (handles all text + photos) —————

async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user = update.effective_user
    state = context.user_data.get("state")

    # — Study request —
    if state == "study_request":
        text = update.message.text or ""
        if not text.strip():
            await update.message.reply_text("Please type your study material requ
            return

        conn = db_pool.getconn()
        try:
            cur = conn.cursor()
            cur.execute(
                "INSERT INTO study_requests (user_id, username, full_name, reques
                (user.id, user.username or "", user.full_name or "", text)
            )
            req_id = cur.fetchone()[0]
            conn.commit()
        finally:
            cur.close()
            db_pool.putconn(conn)

        await context.bot.send_message(

```

```

        ADMIN_ID,
        f"New Study Material Request (ID: {req_id})\n"
        f"From: {user.full_name} (@{user.username or 'no username'}) ID: {use
        f"Request: {text}\n\n"
        f"Reply with: /reply {user.id} <your message>"
    )
    context.user_data["state"] = None
    await update.message.reply_text(
        "Your request has been sent to admin. We will get back to you soon!\n"
        "You can also contact admin directly:",
        reply_markup=InlineKeyboardMarkup([
            [InlineKeyboardButton("📞 WhatsApp Admin", url=f"https://wa.me/{A
            [InlineKeyboardButton("🏠 Main Menu", callback_data="back_main")]
        ])
    )
    return

# — Checker quantity —
if state == "checker_qty":
    text = update.message.text or ""
    try:
        qty = int(text.strip())
        if qty < 1:
            raise ValueError
    except ValueError:
        await update.message.reply_text("Please enter a valid number (e.g. 1,
        return

checker_type = context.user_data.get("checker_type")
total = qty * CHECKER_PRICE

context.user_data["checker_qty"] = qty
context.user_data["checker_total"] = total
context.user_data["state"] = "checker_payment"

await update.message.reply_text(
    f"Order Summary:\n"
    f"Type: {checker_type}\n"
    f"Quantity: {qty}\n"
    f"Total: GH\u00a2{total:.2f}\n\n"
    f"Make payment to:\n"
    f"MoMo Number: {MOMO_NUMBER}\n"
    f"Name: {ACCOUNT_NAME}\n"
    f"Amount: GH\u00a2{total:.2f}\n\n"
    f"After payment, send a screenshot of your payment here."
)
return

```

```

# — Payment screenshot (checker or form) —
if state in ("checker_payment", "form_payment"):
    if not update.message.photo:
        await update.message.reply_text(
            "Please send a screenshot (photo) of your payment."
        )
    return

file_id = update.message.photo[-1].file_id

# Get referrer from DB
user_record = get_user(user.id)
referrer_id = user_record[3] if user_record else None

if state == "checker_payment":
    checker_type = context.user_data.get("checker_type")
    qty = context.user_data.get("checker_qty", 1)
    total = context.user_data.get("checker_total", CHECKER_PRICE)

    order_id = create_order(
        user_id=user.id,
        username=user.username or "",
        full_name=user.full_name or "",
        order_type="checker",
        checker_type=checker_type,
        quantity=qty,
        total=total,
        file_id=file_id,
        referrer_id=referrer_id
    )

    admin_text = (
        f"New Checker Order #{order_id}\n"
        f"Customer: {user.full_name} (@{user.username or 'no username'})\n"
        f"ID: {user.id}\n"
        f"Type: {checker_type}\n"
        f"Quantity: {qty}\n"
        f"Total: GH\u00a2{total:.2f}\n"
        f"Referrer: {referrer_id or 'None'}"
    )
else:
    university = context.user_data.get("university")
    price = context.user_data.get("form_price", FORM_PRICE_PUBLIC)

    order_id = create_order(
        user_id=user.id,

```



```

        username=user.username or "",
        full_name=user.full_name or "",
        order_type="form",
        university=university,
        total=price,
        file_id=file_id,
        referrer_id=referrer_id
    )

    admin_text = (
        f"New Form Order #{order_id}\n"
        f"Customer: {user.full_name} (@{user.username or 'no username'})\n"
        f"ID: {user.id}\n"
        f"University: {university}\n"
        f"Total: GH\u00a2{price:.2f}\n"
        f"Referrer: {referrer_id or 'None'}"
    )

    approve_btn = InlineKeyboardButton("Approve", callback_data=f"approve_{order_id}")
    reject_btn = InlineKeyboardButton("Reject", callback_data=f"reject_{order_id}")

    await context.bot.send_photo(
        ADMIN_ID,
        photo=file_id,
        caption=admin_text,
        reply_markup=InlineKeyboardMarkup([[approve_btn, reject_btn]])
    )

    context.user_data["state"] = None
    await update.message.reply_text(
        "Payment screenshot received! Admin will verify and approve shortly.\n"
        "You will be notified here once approved."
    )
    return

# — Withdrawal MoMo number —
if state == "withdraw_momo":
    momo = update.message.text.strip() if update.message.text else ""
    if not momo:
        await update.message.reply_text("Please enter your MoMo number.")
        return

    amount = context.user_data.get("withdraw_amount", 0)
    user_record = get_user(user.id)
    balance = float(user_record[4]) if user_record else 0

    if amount > balance:

```

```

        await update.message.reply_text(
            f"Insufficient balance. Your current balance is GH\u00a2{balance:}
        )
        context.user_data["state"] = None
        return

    withdrawal_id = create_withdrawal(user.id, amount, momo)

    await context.bot.send_message(
        ADMIN_ID,
        f"Withdrawal Request #{withdrawal_id}\n"
        f"From: {user.full_name} (@{user.username or 'no username'}) ID: {use
        f"Amount: GH\u00a2{amount:.2f}\n"
        f"MoMo: {momo}\n\n"
        f"Please send payment manually."
    )

    context.user_data["state"] = None
    await update.message.reply_text(
        f"Withdrawal request of GH\u00a2{amount:.2f} submitted!\n"
        f"Admin will process your payment to {momo} shortly.",
        reply_markup=main_menu_keyboard()
    )
    return

# Default
await update.message.reply_text(
    "Use the menu to get started:",
    reply_markup=main_menu_keyboard()
)

# — Admin Approval / Rejection —————

async def admin_action_callback(update: Update, context: ContextTypes.DEFAULT_TYP
    query = update.callback_query

    # Admin check FIRST
    if query.from_user.id != ADMIN_ID:
        await query.answer("You are not authorized.", show_alert=True)
        return

    await query.answer()
    data = query.data # approve_123 or reject_123

    action, order_id_str = data.split("_", 1)
    order_id = int(order_id_str)

```

```

order = get_order(order_id)
if not order:
    await query.edit_message_caption("Order not found.")
    return

# order columns: id, user_id, username, full_name, order_type, checker_type,
#                university, quantity, total, status, file_id, referrer_id, c
(oid, user_id, username, full_name, order_type, checker_type,
 university, quantity, total, status, file_id, referrer_id,
 commission_paid, created_at, updated_at) = order

if status != "pending":
    await query.edit_message_caption(
        query.message.caption + f"\n\nAlready {status}."
    )
    return

if action == "approve":
    if order_type == "checker":
        # Fetch pins atomically
        pins = fetch_and_lock_pins(checker_type, quantity, order_id)
        if len(pins) < quantity:
            await query.answer(
                f"Not enough pins! Only {len(pins)} available for {checker_ty
                show_alert=True
            )
            return

        approve_order(order_id)

        # Send pins to customer
        pin_text = f"Your {checker_type} checker card(s):\n\n"
        for i, (serial, pin) in enumerate(pins, 1):
            pin_text += f"Card {i}:\nSerial: {serial}\nPIN: {pin}\n\n"
        pin_text += f"Check results at: {RESULT_LINK}"

        try:
            await context.bot.send_message(user_id, pin_text)
            await query.edit_message_caption(
                query.message.caption + "\n\nAPPROVED - Pins sent to customer
            )
        except Exception as e:
            logger.error(f"Failed to send pins to user {user_id}: {e}")
            await query.edit_message_caption(
                query.message.caption + f"\n\nAPPROVED but failed to send pin
            )

```

```

else: # form order
    approve_order(order_id)

    # Notify user
    try:
        await context.bot.send_message(
            user_id,
            f"Your application form for {university} has been approved!\n"
            f"Admin will send your form via WhatsApp or Telegram shortly."
            f"Contact admin if you do not receive it within 1 hour.",
            reply_markup=InlineKeyboardMarkup([
                [InlineKeyboardButton("📞 WhatsApp Admin", url=f"https://
            ]
        )
        await query.edit_message_caption(
            query.message.caption + "\n\nAPPROVED - Customer notified."
        )
    except Exception as e:
        logger.error(f"Failed to notify user {user_id}: {e}")
        await query.edit_message_caption(
            query.message.caption + f"\n\nAPPROVED but failed to notify."
        )

    # Credit affiliate commission for form orders
    if referrer_id:
        add_affiliate_commission(referrer_id, order_id)
        try:
            referrer_record = get_user(referrer_id)
            new_balance = float(referrer_record[4]) if referrer_record else 0
            await context.bot.send_message(
                referrer_id,
                f"You earned GH\u00a225.00 commission!\n"
                f"A form was purchased through your referral link.\n"
                f"Your new balance: GH\u00a2{new_balance:.2f}\n\n"
                f"Go to Affiliate Program to request withdrawal."
            )
        except Exception as e:
            logger.warning(f"Could not notify referrer {referrer_id}: {e}")

elif action == "reject":
    reject_order(order_id)
    try:
        await context.bot.send_message(
            user_id,
            f"Your order #{order_id} was not approved.\n\n"
            f"Please contact admin for assistance:",
            reply_markup=InlineKeyboardMarkup([

```

```

        [InlineKeyboardButton("📞 WhatsApp Admin", url=f"https://wa.r
    ))
    )
    await query.edit_message_caption(
        query.message.caption + "\n\nREJECTED - Customer notified."
    )
except Exception as e:
    logger.error(f"Failed to notify rejected user {user_id}: {e}")

# — Affiliate System —————

async def show_affiliate_menu(query, context: ContextTypes.DEFAULT_TYPE):
    user = query.from_user
    balance, sales, total_referred = get_affiliate_stats(user.id)

    ref_link = f"https://t.me/scholardealsgh_bot?start={user.id}"

    keyboard = InlineKeyboardMarkup([
        [InlineKeyboardButton("💰 Request Withdrawal", callback_data="affiliate_w
        [InlineKeyboardButton("⬅️ Back", callback_data="back_main")],
    ])

    await query.edit_message_text(
        f"Your Affiliate Dashboard\n\n"
        f"Balance: GH\u00a2{float(balance):.2f}\n"
        f"People referred: {total_referred}\n"
        f"Approved sales via your link: {sales}\n"
        f"Commission per form: GH\u00a225.00\n\n"
        f"Your referral link:\n{ref_link}\n\n"
        f"Share this link! When someone buys a university form using your link, y
        reply_markup=keyboard
    )

async def affiliate_withdraw_callback(update: Update, context: ContextTypes.DEFAU
    query = update.callback_query
    await query.answer()

    user_record = get_user(query.from_user.id)
    balance = float(user_record[4]) if user_record else 0

    if balance < 25:
        await query.edit_message_text(
            f"Your balance is GH\u00a2{balance:.2f}.\n\n"
            f"Minimum withdrawal is GH\u00a225.00.\n"
            f"Keep referring to earn more!",
            reply_markup=back_keyboard()
        )

```

```

        return

    context.user_data["withdraw_amount"] = balance
    context.user_data["state"] = "withdraw_momo"

    await query.edit_message_text(
        f"You want to withdraw GH\u00a2{balance:.2f}.\n\n"
        f"Please type your MoMo number (the number to receive payment):"
    )

# ——— Admin Commands —————

def admin_only(func):
    @wraps(func)
    async def wrapper(update: Update, context: ContextTypes.DEFAULT_TYPE):
        if update.effective_user.id != ADMIN_ID:
            return
        return await func(update, context)
    return wrapper

@admin_only
async def cmd_addpin(update: Update, context: ContextTypes.DEFAULT_TYPE):
    args = context.args
    if len(args) != 3:
        await update.message.reply_text("Usage: /addpin <type> <serial> <pin>\nTy
        return
    checker_type = CHECKER_TYPES.get(args[0])
    if not checker_type:
        await update.message.reply_text("Invalid type. Use 1, 2, 3, or 4.")
        return
    add_pin(checker_type, args[1], args[2])
    await update.message.reply_text(f"Pin added for {checker_type}.")

@admin_only
async def cmd_addpins(update: Update, context: ContextTypes.DEFAULT_TYPE):
    args = context.args
    if not args:
        await update.message.reply_text(
            "Usage:\n/addpins <type>\nSERIAL - PIN\nSERIAL - PIN\n\nTypes: 1=WASS
        )
        return

    checker_type = CHECKER_TYPES.get(args[0])
    if not checker_type:
        await update.message.reply_text("Invalid type. Use 1, 2, 3, or 4.")
        return

```

```

text = update.message.text or ""
lines = text.strip().split("\n")[1:] # Skip the /addpins line
added = 0
errors = 0
for line in lines:
    line = line.strip()
    if not line:
        continue
    if " - " in line:
        parts = line.split(" - ", 1)
    elif "-" in line:
        parts = line.split("-", 1)
    else:
        errors += 1
        continue
    if len(parts) != 2:
        errors += 1
        continue
    serial, pin = parts[0].strip(), parts[1].strip()
    add_pin(checker_type, serial, pin)
    added += 1

    await update.message.reply_text(
        f"Done. Added {added} pins for {checker_type}. Errors: {errors}."
    )

@admin_only
async def cmd_listpins(update: Update, context: ContextTypes.DEFAULT_TYPE):
    stock = get_pin_stock()
    if not stock:
        await update.message.reply_text("No pins in stock.")
        return
    msg = "Current pin stock:\n\n"
    for checker_type, count in stock:
        msg += f"{checker_type}: {count} available\n"
    await update.message.reply_text(msg)

@admin_only
async def cmd_orders(update: Update, context: ContextTypes.DEFAULT_TYPE):
    orders = get_pending_orders()
    if not orders:
        await update.message.reply_text("No pending orders.")
        return
    msg = "Pending orders:\n\n"
    for o in orders:
        oid, uid, uname, otype, ctype, uni, qty, total, created = o
        detail = ctype if otype == "checker" else uni

```

```

        msg += f"#{oid} | {otype.upper()} | {detail} | Qty:{qty} | GH\u00a2{total}
    await update.message.reply_text(msg)

@admin_only
async def cmd_reply(update: Update, context: ContextTypes.DEFAULT_TYPE):
    args = context.args
    if len(args) < 2:
        await update.message.reply_text("Usage: /reply <user_id> <message>")
        return
    user_id = int(args[0])
    message = " ".join(args[1:])
    try:
        await context.bot.send_message(user_id, f"Message from ScholarDeals:\n\n{message}")
        await update.message.reply_text("Message sent.")
    except Exception as e:
        await update.message.reply_text(f"Failed to send: {e}")

@admin_only
async def cmd_broadcast(update: Update, context: ContextTypes.DEFAULT_TYPE):
    args = context.args
    if not args:
        await update.message.reply_text("Usage: /broadcast <message>")
        return
    message = " ".join(args)
    users = get_all_users()
    sent = 0
    failed = 0
    for uid in users:
        try:
            await context.bot.send_message(uid, f"ScholarDeals Update:\n\n{message}")
            sent += 1
        except Exception:
            failed += 1
    await update.message.reply_text(f"Broadcast done. Sent: {sent}, Failed: {failed}")

@admin_only
async def cmd_help_admin(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "Admin Commands:\n\n"
        "/addpin <type> <serial> <pin>\n"
        "/addpins <type> (bulk, SERIAL - PIN per line)\n"
        "/listpins - stock count\n"
        "/orders - pending orders\n"
        "/reply <user_id> <message>\n"
        "/broadcast <message>\n"
        "/help - this message\n\n"
        "Types: 1=WASSCE 2=Nov/Dec 3=BECE 4=Placement"
    )

```



```

    )

# ——— Error Handler —————

async def error_handler(update: object, context: ContextTypes.DEFAULT_TYPE):
    logger.error("Exception while handling update:", exc_info=context.error)

# ——— Main —————

def main():
    init_db()

    app = Application.builder().token(BOT_TOKEN).build()

    # Commands
    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("addpin", cmd_addpin))
    app.add_handler(CommandHandler("addpins", cmd_addpins))
    app.add_handler(CommandHandler("listpins", cmd_listpins))
    app.add_handler(CommandHandler("orders", cmd_orders))
    app.add_handler(CommandHandler("reply", cmd_reply))
    app.add_handler(CommandHandler("broadcast", cmd_broadcast))
    app.add_handler(CommandHandler("help", cmd_help_admin))

    # Callback handlers
    app.add_handler(CallbackQueryHandler(main_menu_callback, pattern="^(back_main
    app.add_handler(CallbackQueryHandler(checker_type_callback, pattern="^checker
    app.add_handler(CallbackQueryHandler(forms_category_callback, pattern="^forms
    app.add_handler(CallbackQueryHandler(university_select_callback, pattern="^un
    app.add_handler(CallbackQueryHandler(affiliate_withdraw_callback, pattern="^a
    app.add_handler(CallbackQueryHandler(admin_action_callback, pattern="^(approv

    # Messages (text + photos)
    app.add_handler(MessageHandler(filters.TEXT | filters.PHOTO, handle_message))

    # Error handler
    app.add_error_handler(error_handler)

    logger.info("ScholarDeals bot started.")
    app.run_polling(allowed_updates=Update.ALL_TYPES)

if __name__ == "__main__":
    main()

```